

Project Meridian — Comprehensive Architectural & Technical Manual

Multi-Page System Specification, Django/MySQL Architectural Blueprint, Database Schemas, REST API Contracts, Data Pipelines & Operational Runbooks

Company: Series Tech Limited	Document ID: STL-ENG-MERIDIAN-SPEC-2026-v2.1	Effective Date: July 2026
Project Name: Project Meridian	Lead Office: Hyderabad HQ	Target Domain: Flagship Django E-Commerce Platform

1. Executive Business Objectives & Market Strategy

Project Meridian represents Series Tech Limited's flagship multi-tenant B2B and B2C E-Commerce enterprise platform. Engineered primarily for retail brands, local artisan networks, and enterprise distributors, Project Meridian solves complex digital supply chain challenges by unifying multi-seller onboarding, inventory synchronization, order fulfillment state machines, and real-time payment settlement into a single, scalable ecosystem.

The core strategic goals of Project Meridian include reducing merchant onboarding friction, guaranteeing sub-second response times for high-throughput product catalog queries, ensuring zero-loss transactional consistency on high-volume checkout surges, and providing an end-to-end transparent platform for local artisans and sellers to manage payouts and stock replenishments.

2. High-Level Architecture & Multi-Tier Ecosystem

Project Meridian follows a Clean Architecture design pattern layered on Python 3.11+ and Django 4.2 LTS, paired with Django REST Framework (DRF) for decoupled web portal integrations, mobile app interfaces, and third-party marketplace webhooks.

Architectural Layer	Enterprise Component & Implementation Details
Backend Core Framework	Python 3.11+ / Django 4.2 LTS with Async ORM, custom security middleware for header sanitization, and DRF viewsets.
Database & Relational Persistence	MySQL 8.0 Engine with InnoDB storage, foreign key integrity constraints, index-optimized search fields, and ProxySQL connection pooling.
Identity & Access Management (IAM)	Custom AbstractUser model with JWT (JSON Web Token) pair rotation, granular Role-Based Access Control (RBAC) for Admins, Artisans/Sellers, and Customers.
Caching & Asynchronous Processing	Redis 7.0 for session management and query result caching; Celery worker clusters for automated PDF invoice generation, email notifications, and inventory updates.

Frontend & User Portals

Django Jinja2 template engine for server-side HTML rendering, complemented by React 18 frontend components for cart state management and interactive catalog filters.

Cloud Media & Asset Storage

AWS S3 / Azure Blob Storage integration managed via django-storages, protected by Signed CloudFront/CDN URLs for high-resolution product imagery and merchant verification documents.

3. In-Depth Database Schema & MySQL ERD Architecture

Project Meridian relies on a normalized MySQL 8.0 database schema designed to handle complex relational hierarchies while maintaining strict data integrity. Key database models and relational mapping include:

- **CustomUser Model:** Extends Django's AbstractUser to incorporate phone numbers, multi-factor authentication (MFA) secrets, default shipping addresses, and a role field designating CUSTOMER, SELLER_ARTISAN, or SYSTEM_ADMIN.
- **SellerProfile Model:** Encapsulates store details for onboarded artisans and vendors, including business registration number, tax ID, bank payout account details, verification status, and commission tier.
- **Category Model:** Hierarchical self-referencing model (parent-child relationship) enabling multi-level category navigation (e.g., Clothing → Men's → Handcrafted Jackets). Indexed on slug and active status.
- **Product Model:** Stores core product metadata including SKU, title, slug, price, cost_price, inventory_count, seller_id foreign key, and full-text search indexes on name and description.
- **ProductVariant Model:** Handles product variations such as color, size, and material, allowing independent pricing overrides and stock tracking per variation.
- **Cart & CartItem Models:** Links custom user sessions or authenticated users to active cart items, automatically computing tax rates, coupon discounts, and shipping charges.
- **Order & OrderItem Models:** Captures snapshot data at time of purchase to protect against retroactive price changes. Tracks status using a strict State Machine: PENDING → PAID → PROCESSING → SHIPPED → DELIVERED → CANCELLED.
- **PaymentTransaction Model:** Stores tokenized payment logs, gateway reference IDs (Stripe transaction ID / Razorpay payment ID), webhook event payloads, and refund tracking.

4. REST API Endpoint Architecture & Contracts

All external client applications, mobile apps, and frontend web portals interface with Project Meridian through standardized Django REST Framework (DRF) endpoints. API design follows strictly RESTful conventions with JSON payloads, standard HTTP response codes, and JWT authentication header headers (Authorization: Bearer <token>).

Endpoint Path	HTTP Verb	Permissions / RBAC	Functional Description
/api/v1/auth/register/	POST	AllowAny	Registers new Customer or Seller; triggers email verification worker.
/api/v1/auth/token/	POST	AllowAny	Authenticates user credentials; returns JWT Access & Refresh token pair.
/api/v1/products/	GET	AllowAny	Lists published catalog items; supports filtering, sorting, and pagination.

<code>/api/v1/seller/products/</code>	POST	IsSeller	Creates new product listing under the authenticated artisan/seller profile.
<code>/api/v1/cart/</code>	GET / POST	IsAuthenticated	Fetches active shopping cart or appends new product items.
<code>/api/v1/checkout/</code>	POST	IsAuthenticated	Validates cart stock, locks inventory, and generates order reference ID.
<code>/api/v1/payments/stripe/webhook/</code>	POST	Stripe Signature	Handles asynchronous payment confirmation webhooks from Stripe.
<code>/api/v1/seller/dashboard/analytics/</code>	GET	IsSeller / IsAdmin	Provides sales metrics, revenue reports, and stock replenishment alerts.

5. Asynchronous Data Pipelines & Celery Task Architecture

To preserve sub-second response times during peak web browsing, intensive computational tasks and external third-party API calls are completely offloaded from HTTP request-response threads into asynchronous Celery background workers backed by a Redis message broker.

- **Order Confirmation & Invoice Generation:** Asynchronously generates itemized PDF invoices using WeasyPrint and dispatches branded order confirmation emails upon receiving payment webhooks.
- **Real-Time Inventory Synchronization:** When an artisan adds or modifies stock, Celery updates search cache keys in Redis and triggers low-stock alerts if inventory falls below the reorder threshold.
- **Seller Payout Calculation Pipeline:** Nightly scheduled Celery Beat cron jobs process artisan sales splits, calculate platform commission fees, and dispatch automated ACH/payout requests.
- **Cart Expiration & Stock Release:** Celery periodically purges abandoned guest carts older than 7 days to release reserved inventory locks back to the active product pool.

6. Detailed Module Specifications

6.1 Artisan & Seller Onboarding Module

Allows artisans and local sellers to apply for store accounts. Features document upload verification (business license, tax certificate) processed via django-storages on secure S3 buckets. System administrators review and approve accounts through custom Django Admin workflows.

6.2 Cart & Checkout Processing Engine

Implements database-level transaction locking (``select_for_update``) during checkout to prevent overselling inventory during high-demand flash sales events. Calculates multi-vendor shipping rates dynamically based on seller origins.

6.3 Payment & Webhook Integration Module

Integrates Stripe and Razorpay payment gateways using tokenized client-side SDKs. The backend validates payment signatures on incoming webhooks to securely update order statuses without relying on client-side confirmation.

7. Engineering Team Roles & Regional Division of Responsibilities

Project Meridian is delivered through a distributed engineering model across Series Tech Limited's regional development centers:

Role / Designation	Regional Center	Core Operational & Technical Responsibilities
Project Lead	Hyderabad HQ	Overall project delivery, sprint planning, client communication, milestone tracking, and risk register management.
Technical Architect	Hyderabad HQ	Clean Architecture design, Django ORM model relationships, MySQL database indexing, and API contract specifications.
Django Backend Engineers	Hyderabad HQ	Development of custom Django apps, REST API viewsets, custom serializers, Celery background tasks, and payment webhooks.
Frontend Engineers	Hyderabad HQ / Pune	Construction of responsive UI templates, React cart state management, and storefront filtering interfaces.
QA Engineers	Chennai Office	Automated API endpoint testing using PyTest, Postman regression suites, and Selenium E2E checkout validation.
DevOps Lead	Bangalore Office	Docker containerization, Gunicorn WSGI configuration, Nginx reverse proxy, CI/CD pipelines, and cloud hosting infrastructure.

8. Deployment Process, Quality Gates & Release Runbooks

Production deployments for Project Meridian follow automated CI/CD pipeline triggers on the main release branch. Before code can be merged into production, the release candidate must successfully pass three mandatory quality gates:

- **Automated Test Gate:** Comprehensive execution of unit and integration tests written in PyTest, achieving a minimum code coverage threshold of 85%.
- **Security Audit Gate:** Code analysis using Bandit to detect Python security risks (e.g., hardcoded secrets, SQL injection vectors, unsafe deserialization).
- **Stakeholder Approval Gate:** Dual sign-off required from the Project Lead (Hyderabad) and the Client Product Owner following staging UAT approval.

9. Post-Go-Live Operational Support & Escalation SLA

After production go-live, Project Meridian transitions into operational support managed by a rotating on-call engineering team. Incidents are categorized by severity:

- **Severity 1 (P1):** Critical outages (e.g., payment processing failure, database deadlock). Immediate 15-minute response SLA; direct escalation to Project Lead and DevOps Lead.
- **Severity 2 (P2):** Partial feature impairment (e.g., image upload delay, report export error). 2-hour response SLA; resolved by backend engineering team.
- **Severity 3 (P3):** Minor cosmetic or non-blocking issues. Resolved in the next scheduled bi-weekly sprint release.